Kravspecifikation

Document version: 1.0

Projekt Navn: *Autoflow*

System : *Web Applikation(Værksted + Butik + Tidsbestilling)*

Tech (planlagt) : *ASP.NET (Web API) + Angular + SQL Database*

Navn : *Visar Shabani*

StartDato : *03-03-2026*

autoflow

[Afleveringsopgave]

[Ballerup, 2750]

Indholdsbetegnelse

Kravspecifikation..... 1.
Indholdsbetegnelse..... 2.
Indledning.....3.
Kravene..... 3.
Funktionelle Krav.....4.
Administrator funktioner..... 4.
Guest funktioner.....4.
Bruger (User) funktioner.....5.
UseCase Diagram (Administrator,Guest,Bruger) funktioner..... 6.
Ikke-funktionelle Krav..... 7.
Implementering ,teknik og teknologier.....8.
Klasseopbygning..... 9.
Backend – ASP.NET..... 9.
Klassediagram (Backend – ASP.NET) ..... 9.
Frontend – Angular.....10.
Komponentdiagram (Frontend – Angular).....10.
Databasen..... 11.
Sikkerhed.....12.
Design..... 13.
Udviklingsfasen og Test.....14.

Indledning

Autoflow er en webplatform udviklet til et bilværksted, der ønsker at samle sine services i en digital løsning. Autoflow vil blive udviklet som en webplatform for et bilværksted, der sælger biler, reservedele og tilbyder booking af reparationsservice.

Platformen giver kunderne mulighed for at browse og købe biler, bestille reservedele samt booke tider til reparation og andre værkstedsydelser alt sammen online og uden behov for personlig henvendelse.

Målet med projektet er at samle services som virksomhedens tilbyder i en digital løsning, så brugere eller kunder nemt kan finde de nødvendige oplysninger, købe produkter og booke aftaler online.

Projektet udvikles som en fullstack webapplikation med Angular som frontend framework og ASP.NET Web API som backend, kombineret med en SQL Server database. Disse teknologierer valgt på baggrund af personligt erfaring med teknologierne samt deres udbredte anvendelse blandt store programmerings firma i hele verden.

Kravene

For at Autoflow kan fungere som en sammenhængende og professionel webplatform, er der defineret krav til systemet. Disse krav er opdelt i funktionelle og ikke funktionelle krav for hele udviklingsprocessen.

De funktionelle krav beskriver konkret, hvad systemet skal kunne gøre altså hvilke handlinger og funktioner der skal være tilgængelige for systemets tre brugertyper: Administrator, Bruger og Gæst. Hver brugertype har sit eget rettigheder og muligheder, som er tilpasset deres rolle i systemet.

De ikke funktionelle krav beskriver derimod, hvordan systemet skal fungere herunder krav til sikkerhed, brugervenlighed, tilgængelighed og skalerbarhed. Disse krav er lige så vigtige som de funktionelle, da de sikrer, at systemet er stabilt, sikkert og behageligt at bruge.

Samlet set skal webapplikationen som minimum indeholde følgende funktioner og egenskaber, som beskrives i de efterfølgende afsnit.

Funktionelle Krav:

Administrator funktioner

- *Oprette, redigere og slette biler til salg*
- *Oprette, redigere og slette reservedele*
- *Se alle bookinger i systemet*
- *Se ordre*
- *Lagerstyring*

Administrator skal være logget ind for at få adgang til disse funktioner.

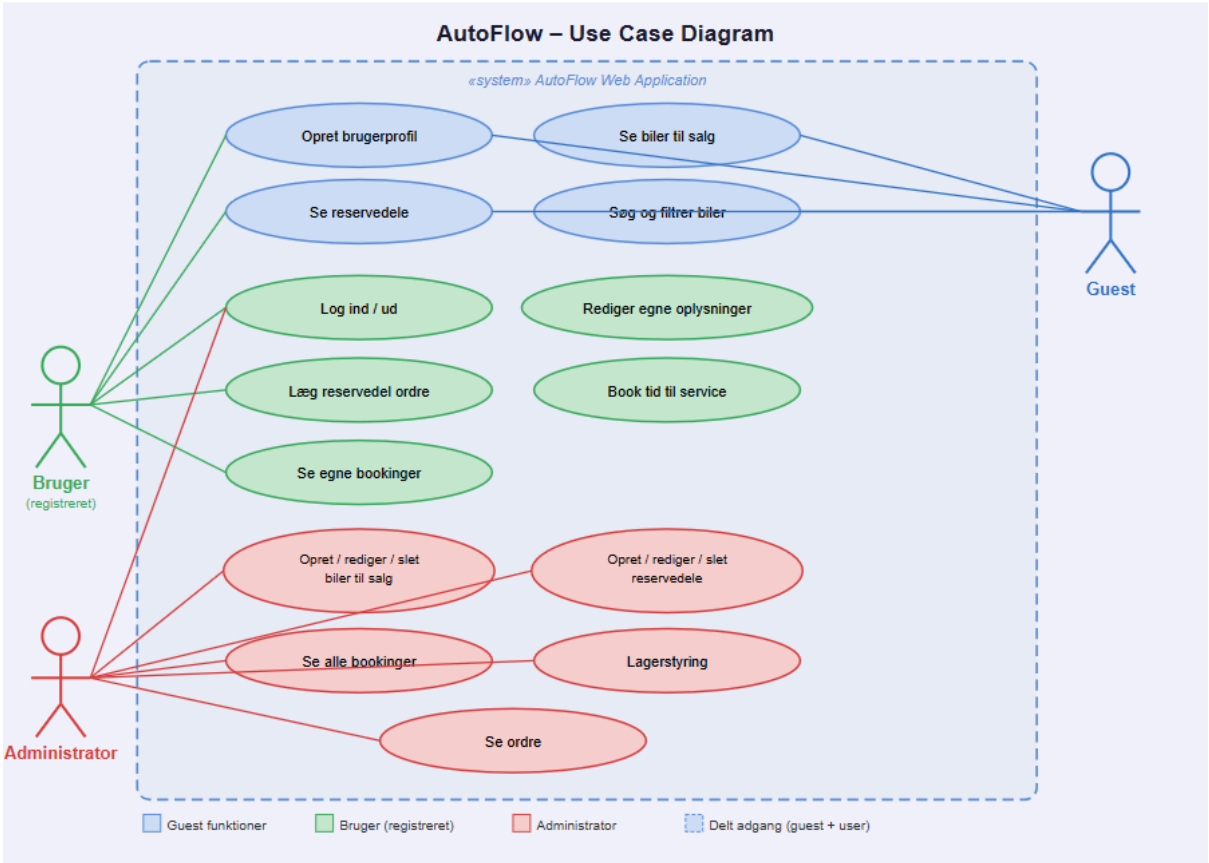
Guest funktioner

- *Oprette brugerprofil*
- *Se biler til salg*
- *Søge og filtrere biler*
- *Se reservedele*

Bruger (User) funktioner

- *Oprette brugerprofil*
- *Logge ind og ud*
- *Redigere egne oplysninger*
- *Se biler til salg*
- *Søge og filtrere biler*
- *Se reservedele*
- *Læge reservedel ordre*
- *Booke tid til service, reparation, lakering eller polering*
- *Se egne bookinger*

UseCase Diagram (Administrator,Guest,Bruger) funktioner



Ikke-funktionelle Krav

Kategori	Krav
Sikkerhed	Adgangskoder hashes med BCrypt og saltes i backend gennem asp.net. Rollebaseret adgangsstyring :(Admin / Bruger / Gæst) håndhæves på serversiden.
Brugervenlighed	Brugergrænsefladen skal være intuitiv og nem at navigere uden forvirring. Responsivt design tilpasset for både brugeren og administratoren.
Tilgængelighed	Webapplikationen skal kunne tilgås via alle moderne browsere (Chrome, Edge).
Skalerbarhed	Applikationen skal gøre det muligt at tilføje nye funktioner uden større omstrukturering.
Dataintegritet	Sletning af biler eller reservedele produkter må ikke bryde eksisterende relationer , og ingen dobbeltbookinger på samme tidspunkt.
Vedligeholdelse	Koden skal være struktureret efter best practices for Angular og ASP.NET.

Implementering ,teknik og teknologier

Systemet udvikles som en fullstack løsning med følgende teknologier:

Frontend:

- *Angular (TypeScript)*
- *Komponentbaseret arkitektur*
- *Routing*
- *Reactive Forms*
- *HTTP Client til API kald*

Backend:

- *ASP.NET Web API*
- *C#*
- *API struktur*
- *JWT Authentication*
- *Role based authorization*

Grunden til at jeg har valgt disse teknologier til backend og frontend for mit projekt er at fordi jeg personligt har erfaring med Angular og c# og ASP.NET, og at idag er Angular og ASP.NET et af de meste anvendte frameworks blandt web udvikler, og det ikke tilfældigvis at virksomheder som Youtube,Google,Reddit og Instagram De bruger angular som framework.

Kommunikationen mellem frontend og backend foregår via HTTPS og JSON.

Systemet opbygges efter en arkitektur, hvor:

- *Controllers håndterer HTTP requests*
- *Services i frontend håndterer kommunikation mellem frontend og backend*
- *Repositories håndterer databaseadgang*

Klasseopbygning

Backend – ASP.NET

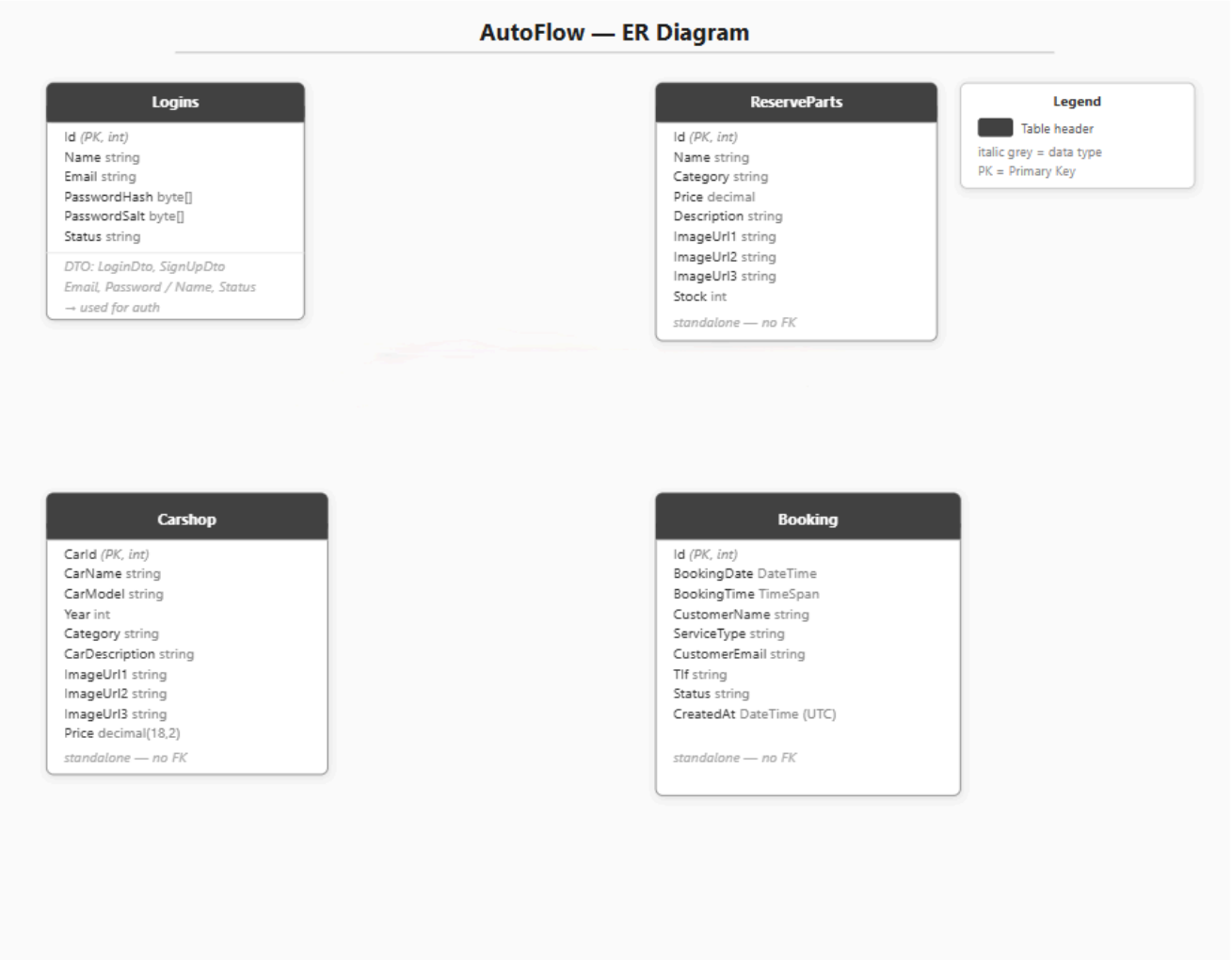
Backend struktureres efter principper fra MVC og Clean Architecture.

- ***Models** repræsenterer databasen*
- ***DTOs** bruges til dataoverførsel mellem API og frontend*
- ***Controllers** håndterer requests*
- ***Services** indeholder logik*
- ***Repositories** står for CRUD-operationer*

Denne struktur gør systemet nemmere at udvide og vedligeholde.

Klassediagram (Backend – ASP.NET)

Nedenfor er 4 hovedklasser af modellen sammen med en DTO (Data transfer object) klasse, der bruges til kryptering af login og tilmelding for brugeren.



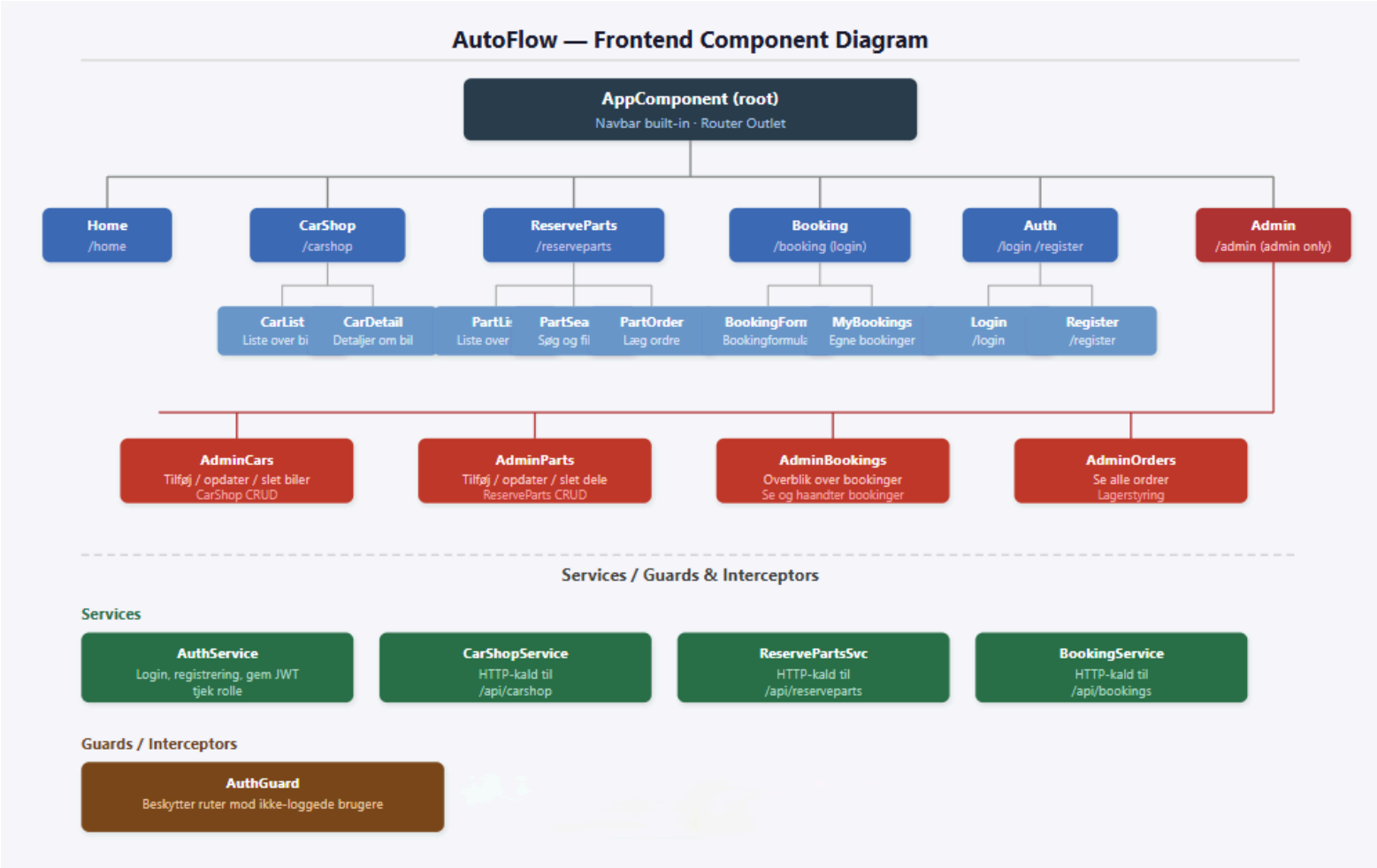
Frontend – Angular

Frontend opdeles i Angular-komponenter:

- *HomeComponent*
- *CarsComponent*
- *CarDetailsComponent*
- *PartsComponent*
- *BookingComponent*
- *AdminDashboardComponent*

Komponentdiagram (Frontend – Angular)

Angular applikationen er opbygget med komponentbaseret arkitektur. Med det sagt nedenfor er et diagram over min komponent struktur :



Databasen

Databasen udvikles i SQL Server og opbygges relationelt.

Primære tabeller:

- *Users*
- *Bil-Shop*
- *Reservedel-Shop*
- *Booking*
- *Ordre*

Databasen designes i tredje normalform for at undgå redundans og sikre dataintegritet.

Relationer:

- *En bruger kan have mange bookinger*
- *En bil kan have flere billeder*
- *En booking tilhører til en bruger*

***Sikkerhed**:*

- *Kryptering af adgangskoden*
- *Brugen af Salt og Hashing*

Systemet skal sikre:

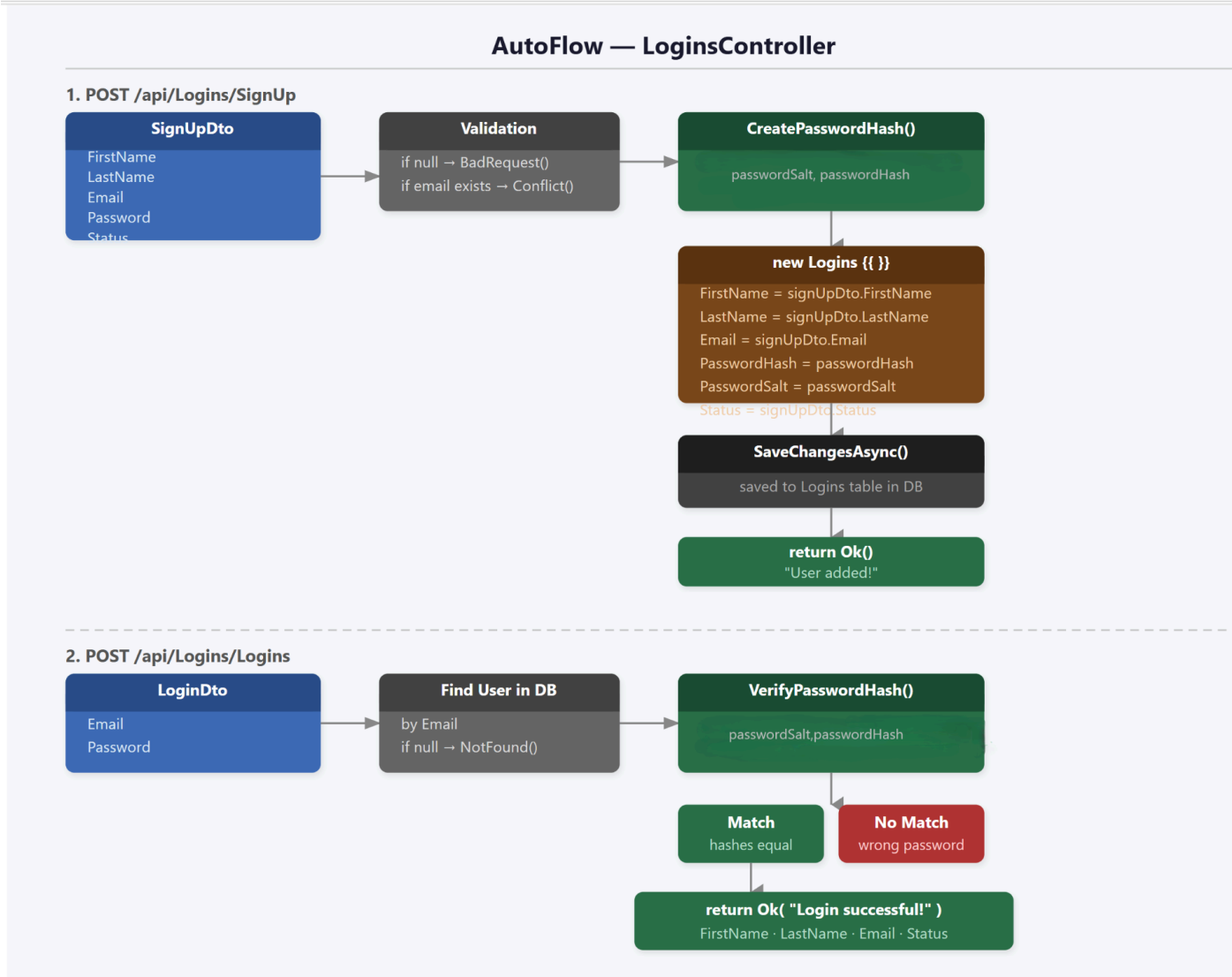
- *Ingen dobbeltbookinger på samme tidspunkt*
- *Sletning af biler eller reservedele må ikke bryde relationer*

Sikkerhed

- *Salting*
- Hashing
- BCrypt

Autoflow anvender sikkerhed for at beskytte brugernes data og sikre, at kun autoriserede brugere kan tilgå beskyttede ressourcer.

For at sikre at koden er beskyttet ,og den kan ikke blive stjålet så bruger jeg salting og hashing med BCrypt i backend. Denne kombination sikrer at koden er krypteret og, kan ikke blive taget gennem rainbow table angreb bliver ineffektive, da hackerens tabeller er lavet uden salt og derfor aldrig vil matche de hashed værdier i databasen.



Design

Designet skal være simpelt og funktionelt.

Brugeren skal nemt kunne:

1. Nemt navigere rundt biler
2. Book tid og få nem adgang til servicetyperne
3. Se både bilmarkedet og reservedelsmarkedet
4. Der skal være tydelig information om virksomheden og kontaktoplysninger

Navigationen skal være tydelig med navigationslinje.

Designprincipper:

- Responsivt layout
- Nemt at navigere fra home page til andre sider
- Nemt at finde det, du leder efter, fra Home page, uanset om det er tids booking , bilmarkedet eller reservedel
- Enkel farvepalette
- Overskuelig bookingformular
- Klar visning af priser og information

Administrationsdelen må være mere funktionel end visuel, men stadig overskuelig.

Udviklingsfasen og Test

Under udviklingen skal kravene testes for at sikre, at systemet lever op til specifikationen.

Testmetoder:

- ***Manuel test***

Med manuelt går jeg selv ind og tester systemet manuelt, ligesom en rigtig bruger ville gøre det,hvor jeg klikker rundt på siden, opretter bookinger, logger ind og ud og tjekker om alt virker som forventet. Det er den mest basale form for test og bruges til at finde fejl og uventede situationer i systemet.

- ***Validering af formularer***

Validering af formularer det handler om at sikre at brugeren indtaster korrekte og gyldige oplysninger i systemets formularer fx ved oprettelse af bruger, booking eller ordre. Eksempler på validering kan være:

- *At email feltet faktisk indeholder en gyldig email*
- *At adgangskoden er lang nok*
- *At et påkrævet felt ikke er tomt*

I Angular håndteres dette typisk med Reactive Forms.

- ***Test af rollebaseret adgang***

Test af rollebaseret adgang hvor jeg tester at de tre brugertyper Administrator, Bruger og Gæst kun har adgang til det de må.

Jeg forventer ikke at være færdig efter den første iteration så nye ting kan blive udviklet under arbejdet. Mindre fejl og forbedringer vil blive håndteret i efterfølgende iterationer.

De mulige fremtidige udvidelser kan være:

- *Online betaling*
- *Email notifikationer*
- *Meget svært men muligheder for brugeren til at sælge biler*